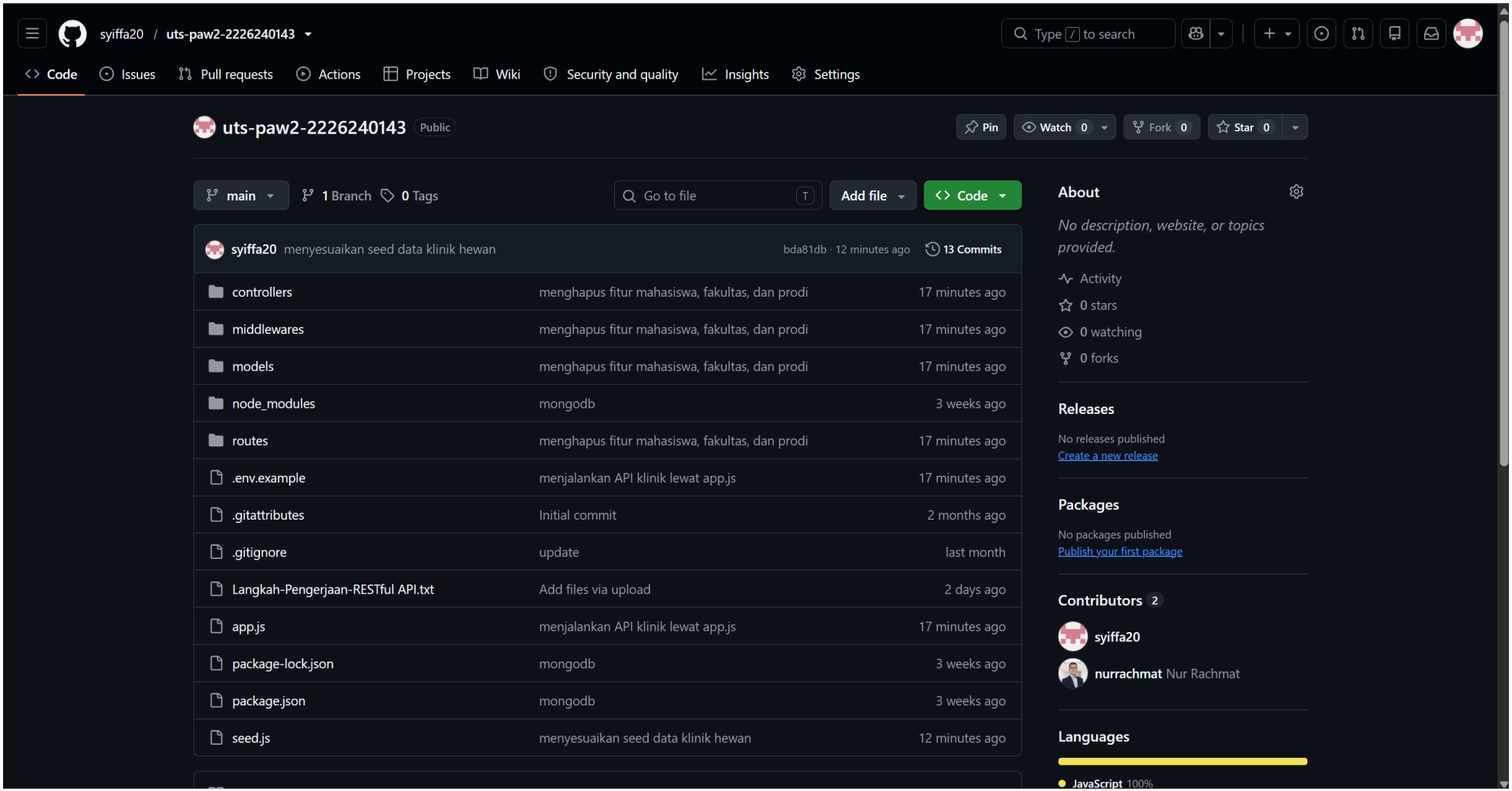
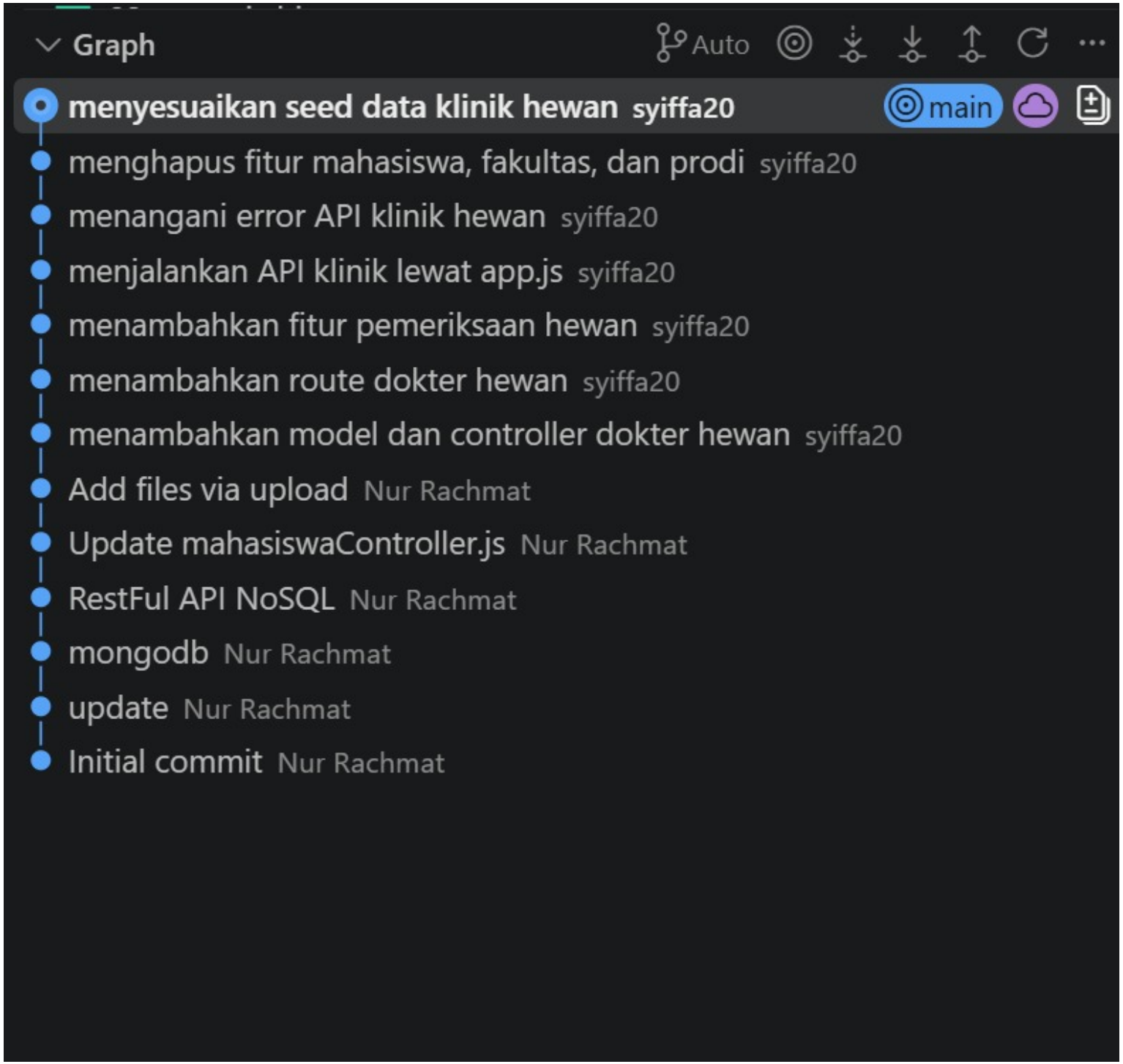
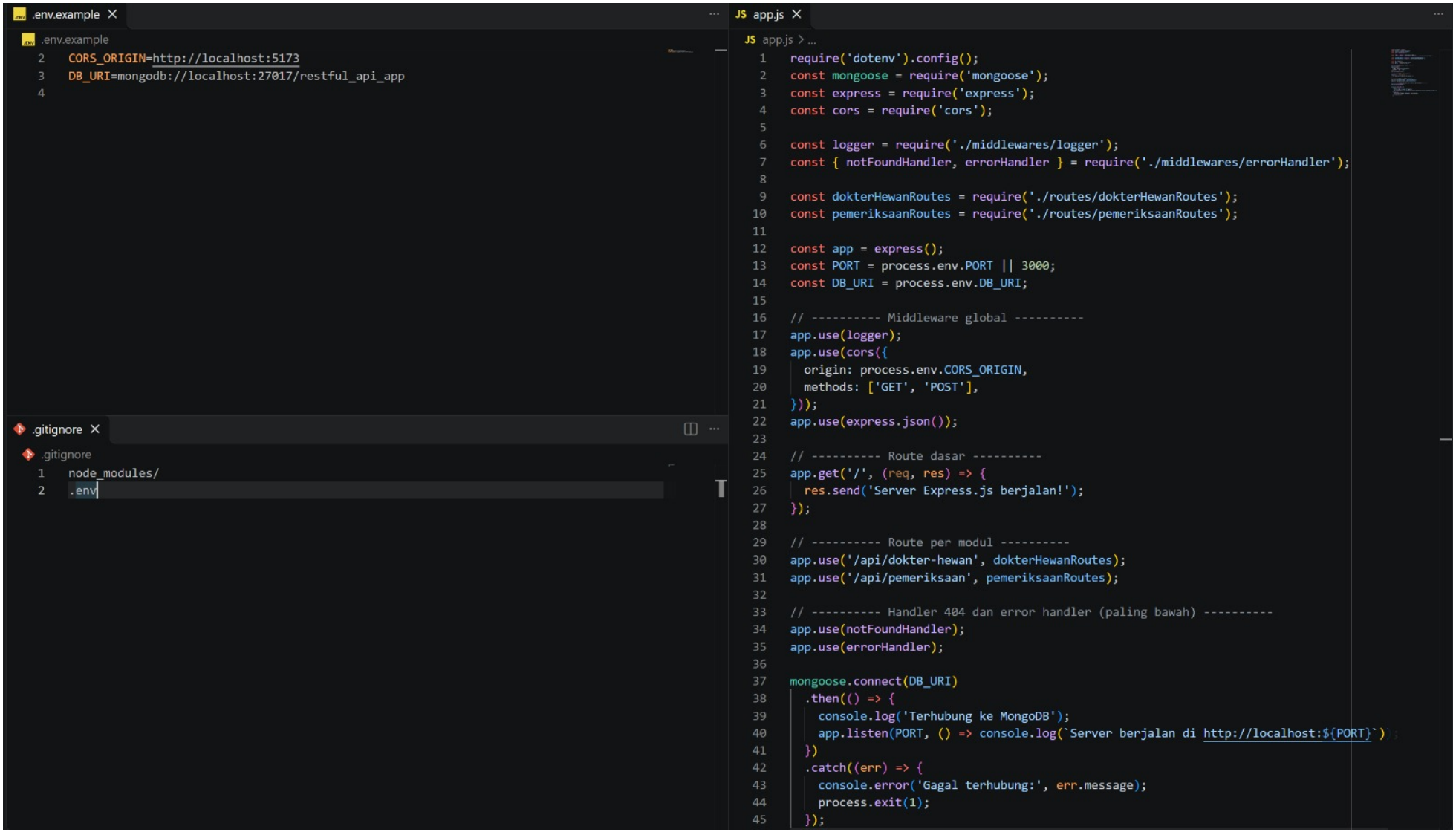
Bukti Repositori — 1/6



Bukti Repositori — 2/6



Bukti Repositori — 3/6



.env.example

```
CORS_ORIGIN=http://localhost:5173
DB_URI=mongodb://localhost:27017/restful_api_app
```

.gitignore

```
node_modules/
.env
```

app.js

```
require('dotenv').config();
const mongoose = require('mongoose');
const express = require('express');
const cors = require('cors');

const logger = require('./middlewares/logger');
const { notFoundHandler, errorHandler } = require('./middlewares/errorHandler');

const dokterHewanRoutes = require('./routes/dokterHewanRoutes');
const pemeriksaanRoutes = require('./routes/pemeriksaanRoutes');

const app = express();
const PORT = process.env.PORT || 3000;
const DB_URI = process.env.DB_URI;

// ---------- Middleware global ----------
app.use(logger);
app.use(cors({
  origin: process.env.CORS_ORIGIN,
  methods: ['GET', 'POST'],
}));
app.use(express.json());

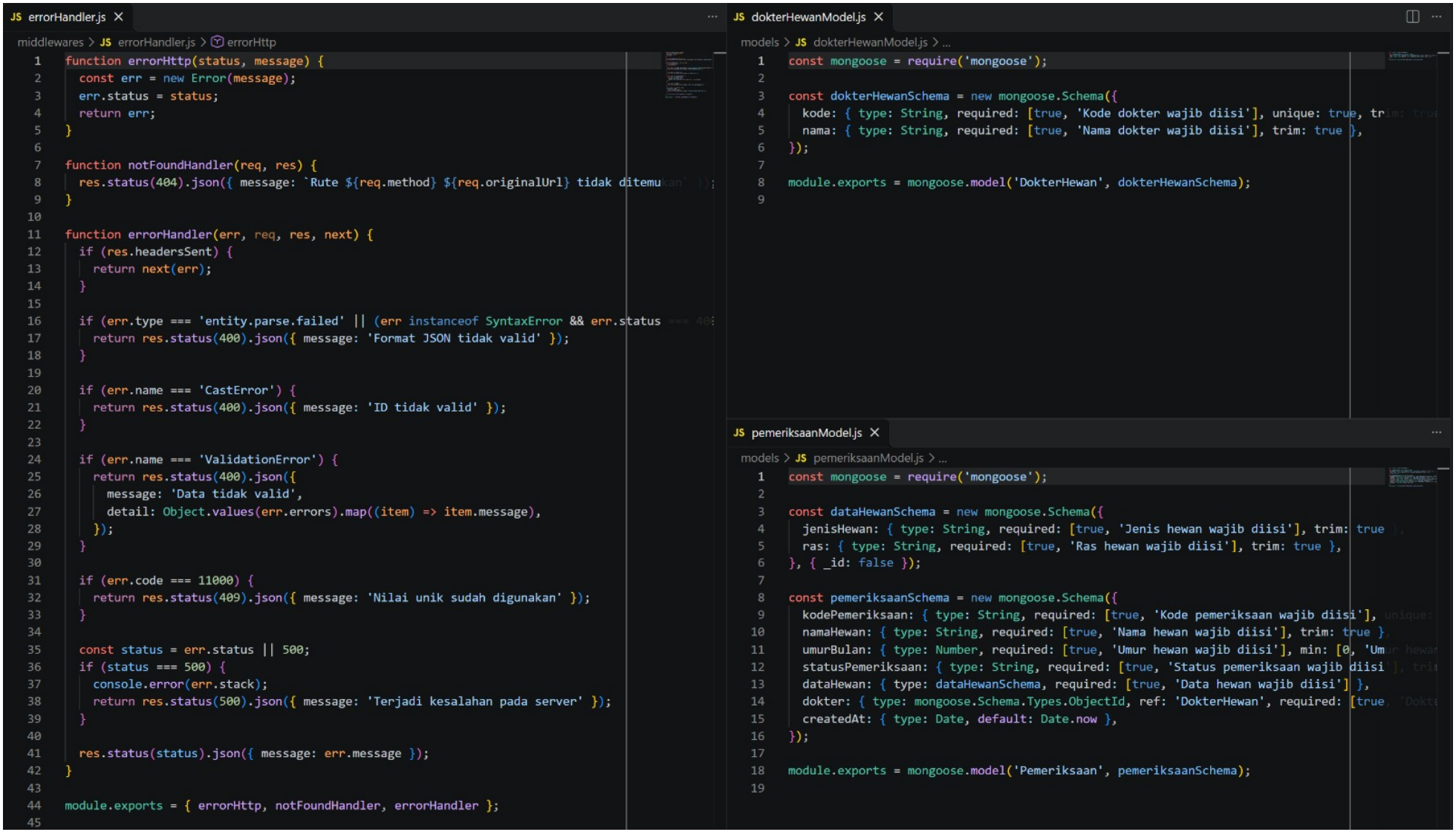
// ---------- Route dasar ----------
app.get('/', (req, res) => {
  res.send('Server Express.js berjalan!');
});

// ---------- Route per modul ----------
app.use('/api/dokter-hewan', dokterHewanRoutes);
app.use('/api/pemeriksaan', pemeriksaanRoutes);

// ---------- Handler 404 dan error handler (paling bawah) ----------
app.use(notFoundHandler);
app.use(errorHandler);

mongoose.connect(DB_URI)
  .then(() => {
    console.log('Terhubung ke MongoDB');
    app.listen(PORT, () => console.log(`Server berjalan di http://localhost:${PORT}`))
  })
  .catch((err) => {
    console.error('Gagal terhubung:', err.message);
    process.exit(1);
  });
```

Bukti Repositori — 4/6



errorHandler.js

middlewares > errorHandler.js > errorHttp

```
function errorHttp(status, message) {
  const err = new Error(message);
  err.status = status;
  return err;
}

function notFoundHandler(req, res) {
  res.status(404).json({ message: `Rute ${req.method} ${req.originalUrl} tidak ditemukan` });
}

function errorHandler(err, req, res, next) {
  if (res.headersSent) {
    return next(err);
  }

  if (err.type === 'entity.parse.failed' || (err instanceof SyntaxError && err.status === 400)) {
    return res.status(400).json({ message: 'Format JSON tidak valid' });
  }

  if (err.name === 'CastError') {
    return res.status(400).json({ message: 'ID tidak valid' });
  }

  if (err.name === 'ValidationError') {
    return res.status(400).json({
      message: 'Data tidak valid',
      detail: Object.values(err.errors).map((item) => item.message),
    });
  }

  if (err.code === 11000) {
    return res.status(409).json({ message: 'Nilai unik sudah digunakan' });
  }

  const status = err.status || 500;
  if (status === 500) {
    console.error(err.stack);
    return res.status(500).json({ message: 'Terjadi kesalahan pada server' });
  }

  res.status(status).json({ message: err.message });
}

module.exports = { errorHttp, notFoundHandler, errorHandler };
```

dokterHewanModel.js

models > dokterHewanModel.js > ...

```
const mongoose = require('mongoose');

const dokterHewanSchema = new mongoose.Schema({
  kode: { type: String, required: [true, 'Kode dokter wajib diisi'], unique: true, trim: true },
  nama: { type: String, required: [true, 'Nama dokter wajib diisi'], trim: true },
});

module.exports = mongoose.model('DokterHewan', dokterHewanSchema);
```

pemeriksaanModel.js

models > pemeriksaanModel.js > ...

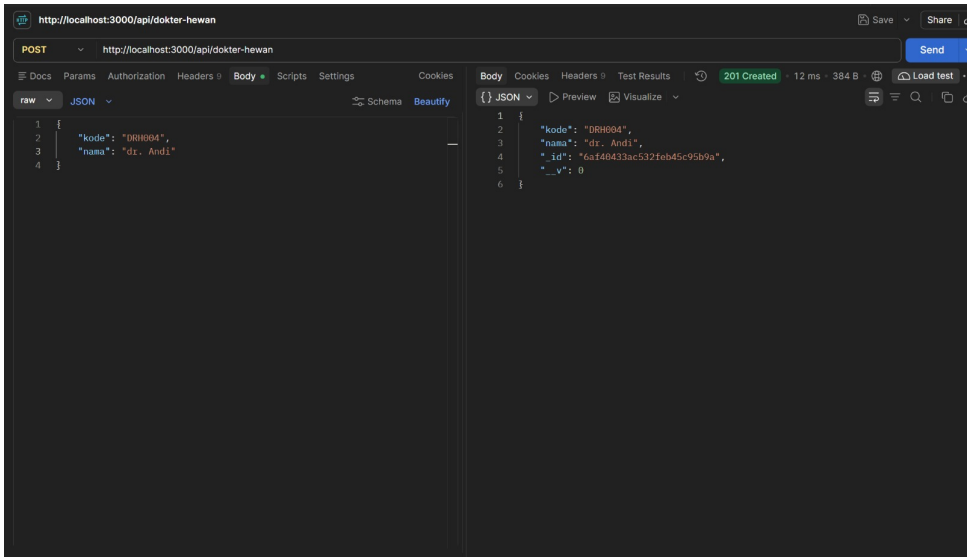
```
const mongoose = require('mongoose');

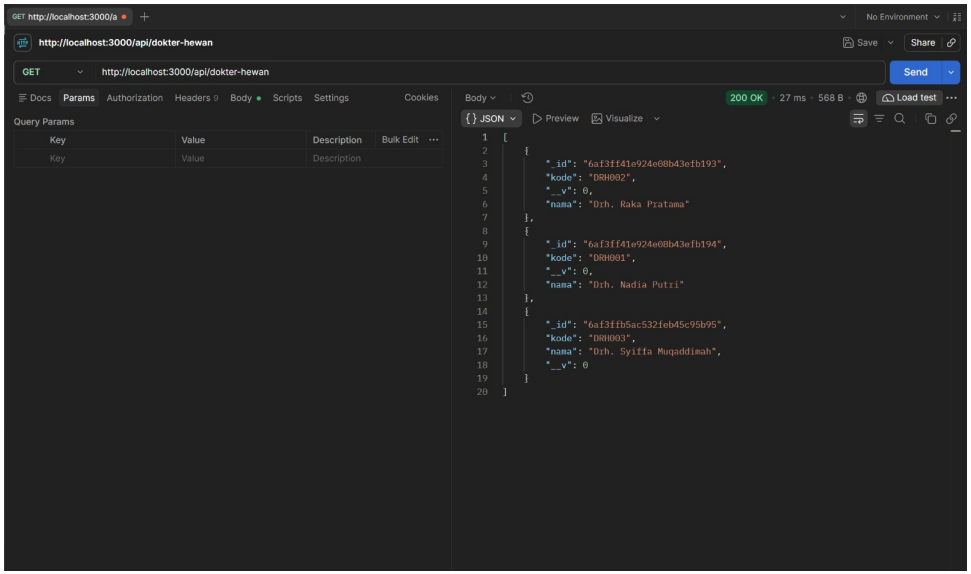
const dataHewanSchema = new mongoose.Schema({
  jenisHewan: { type: String, required: [true, 'Jenis hewan wajib diisi'], trim: true },
  ras: { type: String, required: [true, 'Ras hewan wajib diisi'], trim: true },
}, { _id: false });

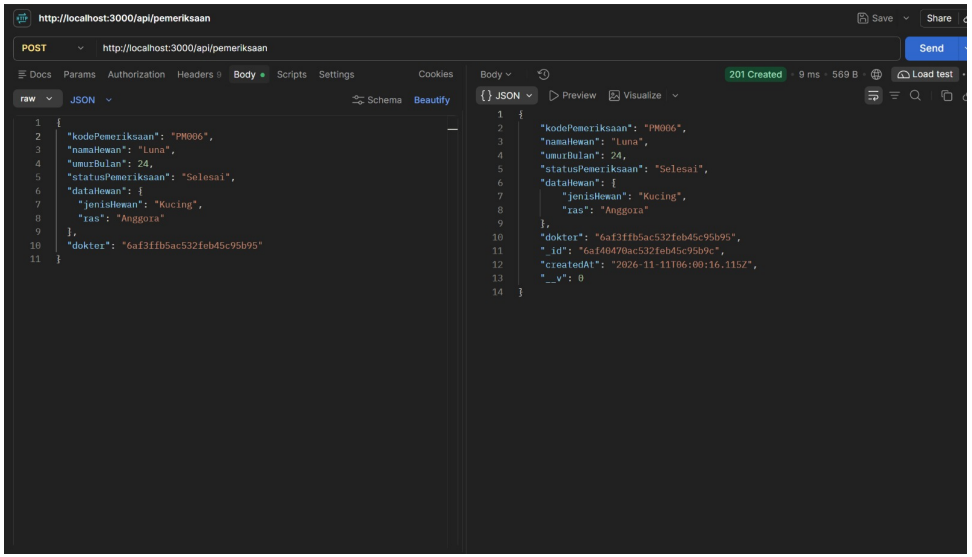
const pemeriksaanSchema = new mongoose.Schema({
  kodePemeriksaan: { type: String, required: [true, 'Kode pemeriksaan wajib diisi'], unique: true, trim: true },
  namaHewan: { type: String, required: [true, 'Nama hewan wajib diisi'], trim: true },
  umurBulan: { type: Number, required: [true, 'Umur hewan wajib diisi'], min: [0, 'Umur hewan tidak boleh negatif'] },
  statusPemeriksaan: { type: String, required: [true, 'Status pemeriksaan wajib diisi'], trim: true },
  dataHewan: { type: dataHewanSchema, required: [true, 'Data hewan wajib diisi'] },
  dokter: { type: mongoose.Schema.Types.ObjectId, ref: 'DokterHewan', required: [true, 'Dokter wajib dipilih'] },
  createdAt: { type: Date, default: Date.now },
});

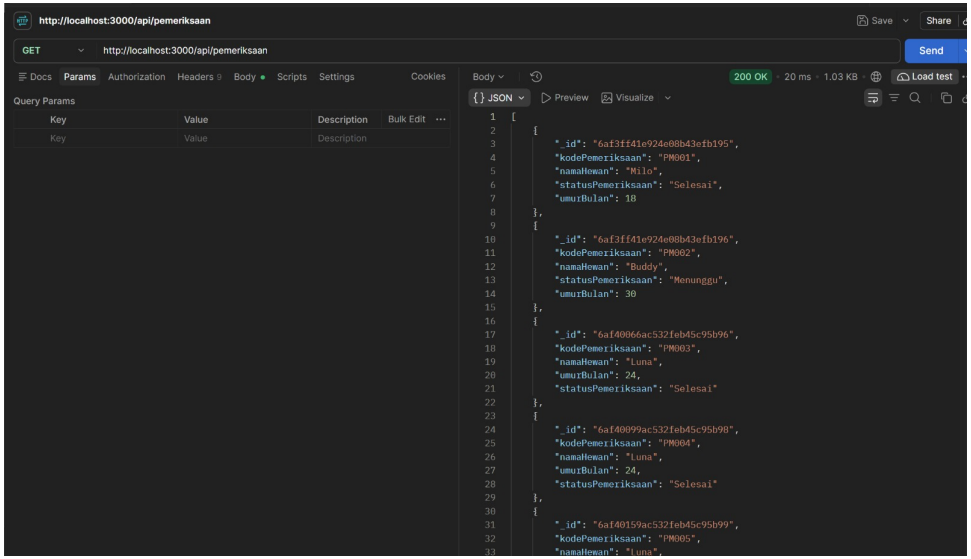
module.exports = mongoose.model('Pemeriksaan', pemeriksaanSchema);
```

Bukti Repositori — 5/6









Bukti Repositori — 6/6

